WazeCargo Pipeline - Execution Order

Step 1: Data Rebuild (modeling/)

03 modeling/03_rebuild_clean_maritime.sh

Runs on: EC2 instance

Rebuilds maritime.clean_maritime_imports and maritime.clean_maritime_exports from scratch on the RDS database. Reads from structured.all_imports and structured.all_exports (loaded by team pipeline).

Truncates both target tables, then re-inserts year-by-year from structured.all_imports/exports using PL/pgSQL loops with comprehensive lookup JOINs:
- lkp_aduanas (customs office names)
- lkp_regiones (region names, exports only)
- lkp_puertos (port names and geographic zones)
- lkp_paises (country and continent names)
- lkp_clausulas (trade clause names and abbreviations)
- lkp_harmonized_system (HS6 product descriptions with HS4 fallback)
- lkp_unidades_medida (unit of measure names and abbreviations)
- lkp_regimen_importacion (import regime names, imports only)
- lkp_tipos_carga (cargo type names)

Filters for maritime transport only (cod_via_transporte = '1') and excludes airport records that slipped through (tipo_puerto <> 'Aeropuerto', nombre_puerto NOT ILIKE 'AEROP%'). Handles dirty data with SPLIT_PART/TRIM before casting. Derives HS2/HS4/HS6 codes from ITEM_SA.

Fetches the RDS password from AWS Secrets Manager and downloads the RDS CA bundle for SSL verification. Runs VACUUM ANALYZE at the end for query optimization.

Step 2: Port Congestion ML (modeling/)

04 modeling/04_ml_congestion.sh -> 04_ml_congestion.py

Runs on: EC2 instance

Port Congestion ML Pipeline (Hybrid Ensemble). Runs 5 internal steps:

(1) Monthly Aggregation - Aggregates imports/exports into ml.port_monthly_agg by port, direction, year, and month. Includes shipment counts, trade values (CIF for imports, FOB for exports), weight, quantity, cargo type counts (general, bulk, refrigerated, container), and diversity metrics (commodity, HS4, country, continent).

(2) Feature Engineering - Builds ml.port_features_indexed with: lags (1, 2, 3, 12 months), rolling means (3 and 12 month windows), YoY growth rates, COVID flags (shock 2020 / rebound 2021 / aftershock 2022), COVID-clean lag substitution (skips COVID years using lag_24/36/48), cargo-type ratios, seasonal indicators (sin/cos encoding), and a composite congestion index (weighted combination of normalized shipments, value, diversity, and cargo type proportions).

(3) Walk-Forward Cross-Validation - Tests 7 models per port-direction pair: Baseline Seasonal Naive (COVID-aware), LightGBM, XGBoost, Random Forest, Ridge, Lasso, ElasticNet. COVID years are excluded from training or down-weighted. Results stored in ml.port_cv_metrics.

(4) Hybrid Model Selection - Big ports (>= 500 avg shipments/month) always use Baseline Seasonal Naive (stable patterns, ML overfits). Small ports use the best ML model by lowest CV MAPE. Stored in

ml.port_model_selection.

(5) 2026 Forecasts - Generates 12-month forecasts per port-direction pair using the selected model. Stored in ml.port_forecast_2026.

Step 3: Commodity ML (modeling/)

05 modeling/05_ml_commodity.sh -> 05_ml_commodity.py

Runs on: EC2 instance

Commodity ML Pipeline. Same 5-step structure as congestion but at a finer granularity: HS2 chapter x port x direction.

(1) Monthly Aggregation - Aggregates into ml.commodity_monthly_agg by HS2, port, direction, year, month. Includes shipment count, trade value, weight, quantity, and dominant product description.

(2) Feature Engineering - Builds ml.commodity_features with the same lag, rolling mean, YoY growth, and COVID-aware feature set as the port pipeline. Partitioned by HS2 x port x direction instead of just port x direction.

(3) Walk-Forward CV - Tests the same 7 models. Only evaluates combinations with >= 36 months of history (MIN_MONTHS filter). Results stored in ml.commodity_cv_metrics.

(4) Model Selection - Selects best model per HS2 x port x direction by lowest CV MAPE. No hybrid override (unlike port pipeline). Stored in ml.commodity_model_selection.

(5) 2026 Forecasts - Only forecasts combinations active in 2024+. Includes commodity descriptions for readability. Stored in ml.commodity_forecast_2026.

WazeCargo Pipeline - Execution Order

Visual Flow Diagram

```
RDS structured.all_imports / all_exports  (from team pipeline)
  |
  v  03_rebuild_clean_maritime.sh
RDS maritime.clean_maritime_imports / exports
  |
  +--> 04_ml_congestion.sh --> ml.port_*       (port-level forecasts)
  |
  +--> 05_ml_commodity.sh  --> ml.commodity_*  (HS2 x port forecasts)
```

Key Notes

Input tables (from team pipeline)

Scripts 00 through 02 (loading raw CSVs to S3, raw-to-staging Parquet conversion, staging-to-RDS load) are maintained by the team in a separate branch. They produce the structured.all_imports and structured.all_exports tables that serve as input to this pipeline.

Shell wrappers (.sh) for ML scripts

Scripts 04 and 05 each have a .sh wrapper that fetches RDS credentials from AWS Secrets Manager, exports them as environment variables (RDS_HOST, RDS_PORT, RDS_USER, RDS_PASSWORD, RDS_DBNAME), and then invokes the corresponding .py script. The .sh files are the intended entry point on EC2.

Database schemas

- structured: Input tables (all_imports, all_exports) and lookup tables (lkp_aduanas, lkp_puertos, lkp_paises, lkp_tipos_carga, lkp_clausulas, lkp_regiones, lkp_regimen_importacion, lkp_unidades_medida, lkp_harmonized_system)
- maritime: Cleaned, enriched maritime-only tables (clean_maritime_imports, clean_maritime_exports)
- ml: All ML outputs - aggregations, feature panels, CV metrics, model selections, and 2026 forecasts

Shared ML utilities

Both ML pipelines import wz_ml_utils (modeling/wz_ml_utils.py) which provides common functions: walk-forward evaluation, forecast generation, sample weight calculation, COVID year constants, and port eligibility filtering.

ML models evaluated

Both pipelines test 7 models: Baseline Seasonal Naive (COVID-aware), LightGBM, XGBoost, Random Forest, Ridge, Lasso, and ElasticNet. Model selection is done via walk-forward cross-validation using MAPE as the metric. COVID years (2020-2022) are excluded from training or handled via clean lag substitution.